

GoHighLevel API Integration

Medellín Social / EasyStreet – v1.0

AUDIENCE	BASE URL	API VERSION	CONTRACT
Backend / GHL implementation	services.leadconnectorhq.com	2021-07-28	18 endpoints tested

No speculative endpoints. Everything in this document was tested directly against the live GHL location. Untested surfaces are listed in **Section 20. No tokens are included in this document.**

Table of Contents

1	Overview & Authentication
2	Quick Endpoint Reference
3	Real Estate Listing — Object
4	Real Estate Listing — API
5	Real Estate Listing — Field Value Rules
6	Associations — Definitions
7	Associations — API
8	Business / Company — Object
9	Business / Company — API
10	Business Custom Fields — API
11	Business Field Value Rules
12	Contact — Realtor Operational Fields
13	Contact — API
14	Contact ↔ Company Relationship
15	Full Relationship Model
16	Field Mapping — Real Estate
17	Field Mapping — Business
18	API Quirks — Read This
19	Test Records
20	Not Included in This Version

1 — Overview & Authentication

Environment

ITEM	VALUE
Base API URL	<code>https://services.leadconnectorhq.com</code>
GHL Location ID	<code>Nk32TtnZAnbsRtIZ0lWS</code>
Private Integration	Medellín Social Website API

Required headers

```
Authorization: Bearer YOUR_PRIVATE_INTEGRATION_TOKEN
Version: 2021-07-28
Accept: application/json
```

For any request with a JSON body, also send:

```
Content-Type: application/json
```

 **The token is never included in this document.** Store it as an environment secret (`GHL_PRIVATE_TOKEN`) on the FastAPI side. Do not commit it, and do not expose it to the browser — all GHL calls must be server-to-server.

Scopes enabled on the private integration

```
objects/record.readonly
objects/record.write

associations.readonly
associations/relation.readonly
associations/relation.write

contacts.readonly
contacts.write

businesses.readonly
businesses.write
```

Calendar scopes are **not** enabled and are out of scope for this version.

A note on response bodies

Every endpoint below was validated end-to-end. Where the exact response shape was not fully catalogued during testing, the document says so explicitly:

Validated successfully; exact response fields may vary.

Build defensively against those responses — read the fields you need, don't assume a fixed schema.

2 — Quick Endpoint Reference

#	PURPOSE	METHOD	PATH
4.1	Search listings	POST	/objects/custom_objects.real_estate_listing/records/search
4.2	Get one listing	GET	/objects/custom_objects.real_estate_listing/records/{recordId}
4.3	Create listing	POST	/objects/custom_objects.real_estate_listing/records
4.4	Update listing	PUT	/objects/custom_objects.real_estate_listing/records/{recordId}?locationId=
7.1	Get association definition	GET	/associations/{associationId}?locationId=
7.2	Get relations for a record	GET	/associations/relations/{recordId}?locationId=
7.3	Create relation	POST	/associations/relations
9.1	List businesses	GET	/businesses/?locationId=
9.2	Get one business	GET	/businesses/{businessId}?locationId=
9.3	Create business	POST	/businesses/
9.4	Update standard business fields	PUT	/businesses/{businessId}?locationId=
10.1	Update business custom fields	PUT	/objects/business/records/{businessId}?locationId=
10.2	Read business custom fields	GET	/objects/business/records/{businessId}?locationId=
13.1	Get contact	GET	/contacts/{contactId}
13.2	Update contact custom fields	PUT	/contacts/{contactId}
14.1	Assign contact to business	POST	/contacts/bulk/business
14.2	Get contacts for a business	GET	/contacts/business/{businessId}?locationId=

3 — Real Estate Listing — Object

A **Real Estate Listing** is a property. It is a dedicated GHL Custom Object.

Terminology: this is *not* a "Business Listing". A Business Listing is a local business profile stored on the Company object (Section 8).

ITEM	VALUE
Object name	Real Estate Listing
Plural	Real Estate Listings
Object key	<code>custom_objects.real_estate_listing</code>
Object ID	<code>6a902152877bcb5cb5a709b5</code>
Primary field	Property Address
Primary field ID	<code>uXcU7A8MXKrCrjDr1b1m</code>
Custom field folder	Real Estate Listing Fields
Folder ID	<code>p8bKXwMn0X9xuB8Xflg8</code>
Total fields	36

Field keys

property_address	listing_title	listing_status
operation	property_category	property_type
municipality	barrio	price
currency	administration_hoa_fee	area_m2
lot_area_m2	bedrooms	bathrooms
parking_spaces	floor	estrato
property_age_years	listing_description	video_url
virtual_tour_url	furnished	pets_allowed
airbnb_allowed	amenities	contact_name
contact_phone	contact_email	preferred_contact_hours
featured	verified	listing_source
due_diligence_declarations	exact_address_public	website_listing_id

 **description** is a reserved key in GHF and could not be used. The property description field key is `listing_description`.

4 – Real Estate Listing – API

All property record keys in `properties` are **suffix-only** (see [Section 18](#)).

4.1 Search / list listings

POST <code>/objects/custom_objects.real_estate_listing/records/search</code>
<pre>{ "locationId": "Nk32TtnZAnbsRtIZ0lWS", "page": 1, "pageLimit": 10 }</pre>

Tested: PASS — valid listing records returned.

⚠ Postman returned **201 Created** for this search call. This is a GHL quirk, not an error. **Treat the JSON response body as authoritative**, not the status code. Do not write a status-code check that only accepts **200** here.

4.2 Get one listing

GET /objects/custom_objects.real_estate_listing/records/{recordId}?locationId=Nk32TtnZAnbsRtIZ0lWS

Tested: 200 OK

Custom field values are returned inside **record.properties** :

```
{
  "properties": {
    "property_address": "...",
    "listing_status": "draft",
    "operation": "sale"
  }
}
```

KEY FORMAT

✓ Correct

property_address

✗ Incorrect

custom_objects.real_estate_listing.property_address

4.3 Create listing

POST /objects/custom_objects.real_estate_listing/records

```
{
  "locationId": "Nk32TtnZAnbsRtIZ0lWS",
  "properties": {
    "property_address": "...",
    "listing_title": "...",
    "listing_status": "draft"
  }
}
```

Tested: 201 Created — created record 6a903aa2584743feeedd490e

✓ For **CREATE**, **locationId** goes in the **JSON body**.

4.4 Update listing

PUT /objects/custom_objects.real_estate_listing/records/{recordId}?locationId=Nk32TtnZAnbsRtIZ0lWS

```
{
  "properties": {
    "listing_status": "approved",
    "price": 625000000,
    "featured": "yes",
    "verified": "yes"
  }
}
```

Tested: 200 OK

⚠ For **UPDATE**, `locationId` **must be in the query string**. Sending it only in the body does not work. This is inconsistent with **CREATE** — it is the single most likely thing to break an integration written from assumption.

CREATE → `locationId` in BODY
UPDATE → `locationId` in QUERY STRING

5 — Real Estate Listing — Field Value Rules

Dropdown values sent to and returned from the API are **internal option keys**, generally lowercase — not the labels shown in the GHL UI.

FIELD	UI LABEL	API VALUE
Listing Status	Draft	<code>draft</code>
Listing Status	Approved	<code>approved</code>
Operation	Sale	<code>sale</code>
Operation	Rent	<code>rent</code>
Property Category	Residential	<code>residential</code>
Property Category	Commercial	<code>commercial</code>
Property Type	Apartment	<code>apartment</code>
Property Type	House	<code>house</code>
Currency	COP	<code>cop</code>
Currency	USD	<code>usd</code>

Yes / No dropdowns

```
yes
no
```

Applies to `furnished`, `pets_allowed`, `airbnb_allowed`, `featured`, `verified`, `exact_address_public`.

Estrato

Returned as a **string**, not a number:

```
"estrato": "5"
```

Amenities

Array of option keys:

```
"amenities": ["elevator", "security", "balcony"]
```

Numeric fields

Sent as **numbers**, not quoted strings:

```
"price": 625000000
```

Applies to `price`, `administration_hoa_fee`, `area_m2`, `lot_area_m2`, `bedrooms`, `bathrooms`, `parking_spaces`, `floor`, `property_age_years`.

6 — Associations — Definitions

Three custom associations exist. Each has a **fixed stored orientation** — `firstRecordId` and `secondRecordId` are not interchangeable.

ASSOCIATION	KEY	ASSOCIATION ID	LINKS	FIRSTRECORDID	SECONDORECORDID
Listing Agent	<code>listing_agent</code>	<code>6a9030a31a03372600cf77c7</code>	Contact ↔ Real Estate Listing	Contact	Listing
Property Owner	<code>property_owner</code>	<code>6a9030a3b1739b9f7dd6262e</code>	Contact ↔ Real Estate Listing	Contact	Listing
Listing Agency	<code>listing_agency</code>	<code>6a9030a3a6611a605762fc43</code>	Business ↔ Real Estate Listing	Business	Listing

Purpose

- **Listing Agent** — links the Realtor Contact who represents the property.
- **Property Owner** — links the Contact who owns the property.
- **Listing Agency** — links the real estate agency Company responsible for the property.

The **record type** always comes **first**, the **listing** always comes **second**. Reversing the order will not produce the relationship you expect.

7 — Associations — API

7.1 Get one association definition

```
GET /associations/{associationId}?locationId=Nk32TtnZAnbsRtIZ0lWS
```

Tested: `200 OK` — validated for all three association IDs.

❌ **Do not use** `GET /associations?locationId=...` — the generic list route returned **404** during testing and is excluded from the integration. Reference the three association IDs from Section 6 as constants in your config instead.

7.2 Get relations for any record

```
GET /associations/relations/{recordId}?locationId=Nk32TtnZAnbsRtIZ0lWS&skip=0&limit=20
```

Tested: `200 OK` for all three record types — Real Estate Listing, Contact, and Business.

This is how you answer "which properties belong to this Realtor?" and "which agency owns this listing?"

Validated successfully; exact response fields may vary.

7.3 Create relation

```
POST /associations/relations
```

```
{
  "locationId": "Nk32TtnZAnbsRtIZ0lWS",
  "associationId": "ASSOCIATION_ID",
  "firstRecordId": "FIRST_RECORD_ID",
  "secondRecordId": "SECOND_RECORD_ID"
}
```

Tested: `201 Created`

USE CASE	ASSOCIATIONID	FIRSTRECORDID	SECONDDRECORDID
Realtor → Listing	6a9030a31a03372600cf77c7	Contact ID	Listing ID
Property Owner → Listing	6a9030a3b1739b9f7dd6262e	Contact ID	Listing ID
Agency → Listing	6a9030a3a6611a605762fc43	Business ID	Listing ID

Duplicate protection

Creating the **same relation twice** returns a **duplicate relation error**. This is **expected behavior**, not a bug.

Backend handling: check existing relations via `7.2` before creating, or catch the duplicate error and treat it as a no-op success. Do not retry blindly.

8 — Business / Company — Object

A **Business / Company** record is the **Local Business Listing**.

Terminology: this is *not* a Real Estate Listing. This object holds cafés, gyms, salons, service providers — and also real estate agencies acting as businesses.

This uses GHF's **standard Business / Company object**, not a custom object.

ITEM	VALUE
Business Object ID	6a356c05d468df1af3af475f
Custom field folder	Medellín Social Business Listing Fields
Folder ID	Jd3JjzdQaISqk4rjG4xp
Custom fields	16

Standard Company fields (reused — do not duplicate)

name	phone	email	website	address
state	city	description	postalcode	country

Custom Business field keys (16)

business_category	business_subcategory	barrio
whatsapp	instagram	google_place_id
price_range	business_listing_status	verified
featured	membership_plan	featured_badge
featured_since	operating_hours	google_rating
active		

Used as **suffix-only keys** inside the Objects API `properties` payload:

KEY FORMAT	
✓ Correct	business_category
✗ Incorrect	business.business_category

9 — Business / Company — API

Standard fields only. Custom fields use a **different API** — see [Section 10](#).

9.1 List businesses

```
GET /businesses/?locationId=Nk32TtnZAnbsRtIZ0lWS&limit=20&skip=0
```

Tested: 200 OK

9.2 Get one business

```
GET /businesses/{businessId}?locationId=Nk32TtnZAnbsRtIZ0lWS
```

Tested: 200 OK

9.3 Create business

```
POST /businesses/
```

```
{
  "locationId": "Nk32TtnZAnbsRtIZ0lWS",
  "name": "Example Business",
  "phone": "+573001112233",
  "email": "business@example.com",
  "website": "https://example.com",
  "address": "Example Address",
  "city": "Medellín",
  "description": "Example Local Business Listing"
}
```

Tested: 201 Created — created business 6a90484230b0bac4a1a61989

9.4 Update standard business fields

```
PUT /businesses/{businessId}?locationId=Nk32TtnZAnbsRtIZ0lWS
```

```
{
  "name": "Updated Business",
  "phone": "+573001112244",
  "description": "Updated through API"
}
```

Tested: 200 OK

10 — Business Custom Fields — API

 The 16 custom Business fields are **not** written through `/businesses/`. They go through the **Objects Record API** with schema key `business`.

10.1 Update business custom fields

```
PUT /objects/business/records/{businessId}?locationId=Nk32TtnZAnbsRtIZ0lWS
```

```
{
  "properties": {
    "business_category": "cafe",
    "business_subcategory": "Specialty Coffee",
    "barrio": "Laureles",
    "whatsapp": "+573001112244",
    "instagram": "https://instagram.com/example",
    "google_place_id": "EXAMPLE_PLACE_ID",
    "price_range": "moderate",
    "business_listing_status": "published",
    "verified": "yes",
    "featured": "yes",
    "membership_plan": "Featured Business",
    "featured_badge": "yes",
    "featured_since": "2026-08-27",
    "operating_hours": "Mon-Fri 8AM-6PM",
    "google_rating": 4.7,
    "active": "yes"
  }
}
```

Tested: 200 OK

10.2 Read business custom fields

```
GET /objects/business/records/{businessId}?locationId=Nk32TtnZAnbsRtIZ0lWS
```

Tested: 200 OK — values returned inside `record.properties`.

Practical implication

A full business write is **two calls**:

1. PUT /businesses/{id}?locationId= → standard fields
2. PUT /objects/business/records/{id}?locationId= → custom fields

Sequence them and treat the pair as one logical transaction on the FastAPI side.

11 — Business Field Value Rules

Business Listing Status

draft	pending_review	approved	published
needs_changes	rejected	unpublished	suspended
expired	archived		

Yes / No fields

`verified` · `featured` · `featured_badge` · `active`

yes
no

Price Range

⚠ Do not assume the UI labels \$, \$\$, \$\$\$, \$\$\$\$ are the API values. They are not.

UI LABEL	API VALUE
\$	budget
\$\$	moderate
\$\$\$	upscale
	luxury
—	not_applicable

Other formats

FIELD	FORMAT	EXAMPLE
business_category	option key	cafe
featured_since	YYYY-MM-DD	2026-08-27
google_rating	number / decimal	4.7
operating_hours	free text	Mon-Fri 8AM-6PM

operating_hours is currently stored as **plain text** in GHL. The website database may convert it to structured JSON later — that conversion belongs on the website side, not in GHL.

12 — Contact — Realtor Operational Fields

The Contact object holds the Realtor Evaluation Survey fields **plus** 6 operational fields. Only the 6 operational fields are in scope here.

FIELD	FIELD ID	FIELD KEY	TYPE
Realtor Level	x62gS7sqmr6EtiveoHtD	contact.realtor_level	Dropdown
Realtor Certified	E2Wcb1WXL3jk2WCk7kTD	contact.realtor_certified	Dropdown
Primary Barrio	nFrE6bYuUP1pWIBak7AC	contact.primary_barrio	Text
Primary Comuna	VAnm1s7fZ0i3BJoYcm0k	contact.primary_comuna	Text
GHL Calendar ID	Od3buJHGVtErpujNKARW	contact.ghl_calendar_id	Text
Member / Community Type	ojJI3M7JTBRBCP4Inn1D	contact.member__community_type	Dropdown

Options

Realtor Level — Level 0 · Level 1 · Level 2 · Level 3

Realtor Certified — Yes · No

Member / Community Type — Realtor · Business Owner · Community Member · Affiliate · Ambassador · Admin

⚠ `contact.member__community_type` has a **DOUBLE UNDERSCORE** between `member` and `community`. A single underscore will not resolve. Copy this key exactly.

GHL Calendar ID exists as a field but **calendar integration is deferred**. Do not build against it in this phase. It is a placeholder pointer only.

13 — Contact — API

⚠ The Contact API uses a **different custom-field format** from the Objects API. It does **not** use `properties`.

13.1 Get contact

```
GET /contacts/{contactId}
```

Tested: 200 OK (test contact `Ulc14o1VlbRgkGV3ffie`)

13.2 Update contact operational fields

```
PUT /contacts/{contactId}
```

Custom fields are sent as an **array of `{id, field_value}` objects**, keyed by **field ID** — not field key:

```
{
  "customFields": [
    { "id": "x62gS7sqmr6EtiveoHtD", "field_value": "Level 1" },
    { "id": "E2Wcb1WXL3jk2Wck7kTD", "field_value": "Yes" },
    { "id": "nFrE6bYuUP1pWIBak7AC", "field_value": "El Poblado" },
    { "id": "VAnm1s7fZ0i3BJoYcm0k", "field_value": "Comuna 14" },
    { "id": "0d3buJHGVtErpujNKARW", "field_value": "TEST-CALENDAR-ID" },
    { "id": "ojJI3M7JTBRCBP4Inn1D", "field_value": "Realtor" }
  ]
}
```

Tested: 200 OK

Note that Contact dropdown values here are the **labels** (`Level 1`, `Yes`, `Realtor`) — unlike the Objects API, which uses lowercase option keys. Keep the two conventions separate in your code.

14 — Contact ↔ Company Relationship

This is a **built-in GHL relationship**, not one of the three custom associations. It connects a business owner Contact to their Company / Local Business Listing.

Business Owner Contact ↔ Company / Local Business Listing

Both endpoints below were validated against the **v3** contact routes.

14.1 Assign contact to business

POST /contacts/bulk/business

```
{
  "locationId": "Nk32TtnZAnbsRtIZ0lWS",
  "ids": ["CONTACT_ID"],
  "businessId": "BUSINESS_ID"
}
```

Tested: 201 Created — response included `success: true`

Note `ids` is an **array**, so multiple Contacts can be attached to one Business in a single call.

14.2 Get contacts for a business

GET /contacts/business/{businessId}?locationId=Nk32TtnZAnbsRtIZ0lWS&limit=25&skip=0

Tested: 200 OK — confirms which Contacts are linked to the Company.

Validated successfully; exact response fields may vary.

15 — Full Relationship Model

CONTACT

- |
- |— Company / Local Business Listing (built-in GHL relationship)
- |
- |— Real Estate Listing as Listing Agent (custom association)
- |
- |— Real Estate Listing as Property Owner (custom association)

COMPANY

- |
- |— Contact(s) (built-in GHL relationship)
- |
- |— Real Estate Listing as Agency (custom association)

REAL ESTATE LISTING

- |
- |— Listing Agent → Contact
- |— Property Owner → Contact
- |— Agency → Company

Backend rule — match on IDs, never on names

 **Do not match records by name, title, or address.** Business names collide, property addresses get re-typed, and people change their display names. Always resolve through stored GHL IDs.

Recommended local ID columns:

```
ghl_contact_id
ghl_business_id
ghl_real_estate_listing_id
```

For Real Estate associations, store a local mapping from **GHL Contact IDs** → **local agent/user records**, so the website can render an agent profile without a round trip to GHL.

16 — Field Mapping — Real Estate → public.listing

GHL FIELD KEY	WEBSITE COLUMN
property_address	direccion_aprox
listing_title	titulo
listing_status	estado
operation	operacion
property_type	tipo_inmueble
municipality	municipio
barrio	barrio
price	precio
currency	moneda
administration_hoa_fee	administracion
area_m2	area_m2
lot_area_m2	area_lote_m2
bedrooms	habitaciones
bathrooms	banos
parking_spaces	parqueaderos
floor	piso
estrato	estrato
property_age_years	antiguedad_anios
listing_description	descripcion
video_url	video_url

GHL FIELD KEY	WEBSITE COLUMN
<code>virtual_tour_url</code>	<code>tour_url</code>
<code>furnished</code>	<code>amoblado</code>
<code>pets_allowed</code>	<code>mascotas</code>
<code>airbnb_allowed</code>	<code>permite_airbnb</code>
<code>amenities</code>	<code>amenidades</code>
<code>contact_name</code>	<code>nombre_contacto</code>
<code>contact_phone</code>	<code>telefono</code>
<code>contact_email</code>	<code>email_contacto</code>
<code>preferred_contact_hours</code>	<code>horario_contacto</code>
<code>featured</code>	<code>destacado</code>
<code>verified</code>	<code>verificado</code>
<code>listing_source</code>	source / business logic
<code>due_diligence_declarations</code>	<code>declaraciones</code>
<code>exact_address_public</code>	<code>mostrar_exacto</code>
<code>website_listing_id</code>	local <code>public.listing.id</code>

Owned by the website, not GHL

geom slug views
search/ranking full photo gallery

Property media lives in `listing_media` / R2, outside GHL entirely. GHL stores only the CRM lifecycle fields.

`website_listing_id` is the **back-reference** — write the local `public.listing.id` into GHL after the first sync so the two systems can reconcile in both directions.

17 — Field Mapping — Business → `public.tiendas`

GHL FIELD	WEBSITE COLUMN
<code>name</code>	<code>nombre</code>
<code>description</code>	<code>descripcion</code>
<code>business_category</code>	<code>categoria</code>
<code>business_subcategory</code>	<code>subcategoria</code>
<code>barrio</code>	<code>barrio_id</code> (requires resolution)
<code>address</code>	<code>direccion</code>
<code>phone</code>	<code>telefono</code>

GHL FIELD	WEBSITE COLUMN
whatsapp	whatsapp
instagram	instagram
website	website
google_place_id	google_place_id
operating_hours	horario
price_range	precio_rango
google_rating	rating_google
active	activo
verified	verificado
featured	destacado
membership_plan	plan
featured_badge	featured_badge
featured_since	featured_desde

`barrio` arrives from GHL as **text** and must be resolved to a `barrio_id` on the website side. Decide the fallback behavior for an unrecognized barrio before go-live.

Owned / derived by the website

lat	lon	foto_url	fotos[]
subido_por	created_at	updated_at	geospatial/search fields

18 — API Quirks — Read This

These are the behaviors that will cost you time if you assume instead of check.

QUIRK

- 1 **Real Estate CREATE** — `locationId` goes in the **JSON body**.
- 2 **Real Estate UPDATE** — `locationId` **must** go in the **query string**. Body-only does not work.
- 3 **Objects API `properties` uses suffix-only keys.** ☒ `listing_status` / ☒ `custom_objects.real_estate_listing.listing_status` . ☒ `business_category` / ☒ `business.business_category` .
- 4 **The Contact API is different.** Custom Contact fields use `customFields: [{ "id": "...", "field_value": "..."}]` , keyed by field ID — not `properties` , not field keys.
- 5 **Dropdown API values ≠ UI labels.** `Published → published` , `Yes → yes` , `$$ → moderate` . Never send the label to the Objects API.
- 6 `contact.member__community_type` has a double underscore.
- 7 `description` is reserved on the Real Estate Listing object. Use `listing_description` .
- 8 `GET /associations?locationId=...` returned 404 and is not validated. Do not use it in production — hardcode the three association IDs.
- 9 **Duplicate association creation returns an error.** Check first, or catch and treat as no-op.
- 10 **The search endpoint returned 201** in Postman. Trust the JSON body, not the status code.
- 11 **Business writes are two calls** — standard fields via `/businesses/` , custom fields via `/objects/business/records/` .
- 12 **Do not store full media in GHL.** Photos live website-side / R2.
- 13 **Do not assume a Contact's role.** Not every Contact is a Realtor or Business Owner. Classify using `contact.member__community_type` and the operational fields, plus your own business logic.

19 — Test Records (QA ONLY)

 **TEST / QA ONLY.** These are not production business data. Do not seed, demo, or migrate against them, and do not let them reach the public website.

PURPOSE	GHL ID
Test Realtor Contact	<code>Ulc14o1VlbRgkGV3ffie</code>
Test Property Owner Contact	<code>iNlezNb78HPkXeFX7oZz</code>
Test Agency Business	<code>6a9032d4d12a04681d3b565b</code>
Original test Real Estate Listing	<code>6a9032d43446ae8ffbb94690</code>
API-created Real Estate Listing	<code>6a903aa2584743feeedd490e</code>
API-created Local Business	<code>6a90484230b0bac4a1a61989</code>

20 — Not Included in This Version

The following are **intentionally excluded**. They are not yet implemented or tested, and nothing in this document should be read as a contract for them.

- » Calendar APIs
- » Appointment booking
- » Free-slot APIs
- » Calendar scopes
- Webhooks
- Approval workflows
- Subscriptions / products
- Offers / deals
- Events
- GHL forms
- Payment APIs
- Communities
- Referral system

Each will be documented separately once implemented and tested.

21 — Recommended Backend Flow

Business submission

```
Website
  ↓
FastAPI
  ↓
create / update Contact  (if required)
  ↓
create / update Business
  ↓
link Contact → Business      [POST /contacts/bulk/business]
  ↓
business_listing_status = pending_review
  ↓
Admin approval in GHL
  ↓
sync published state → PostgreSQL
  ↓
Website displays business
```

Real Estate submission

```
Website / Realtor
↓
FastAPI
↓
create / update Real Estate Listing in GHL
↓
link Listing Agent Contact          [required]
↓
link Property Owner Contact         [optional]
↓
link Agency Business                [optional]
↓
listing_status = pending_review
↓
Admin approval in GHL
↓
FastAPI syncs approved record → public.listing
↓
Website displays property
```

Architectural rule

FastAPI + PostgreSQL is the website-facing read/search layer. Do not call GHL live on every public page request.

GHL is the system of record for CRM lifecycle and approval state. PostgreSQL is the system of record for what the public actually sees. The sync runs on approval events, not on page loads.

22 — Validation Status

FEATURE	STATUS
Real Estate search	✓ Tested
Real Estate get	✓ Tested
Real Estate create	✓ Tested
Real Estate update	✓ Tested
Association definition read	✓ Tested
Association relation read	✓ Tested
Association relation create	✓ Tested
Duplicate relation protection	✓ Tested
Business list	✓ Tested
Business get	✓ Tested
Business create	✓ Tested

FEATURE	STATUS
Business standard update	✓ Tested
Business custom field update	✓ Tested
Business custom field read	✓ Tested
Contact read	✓ Tested
Contact operational field update	✓ Tested
Contact → Company assignment	✓ Tested
Get Company Contacts	✓ Tested
Calendar integration	⏸ Deferred
Webhooks	🟡 Not implemented
Forms / workflows	🟡 Not implemented
Website sync	🟡 Next implementation phase

The core GHL API contract for Contacts, Companies, Real Estate Listings, and their relationships has been validated through direct Postman testing. Edward can now build the FastAPI integration against these tested contracts.